

Vivo Energy 360

*A synthetic end-to-end data engineering platform
for a downstream energy business*

37.2m

FACT ROWS

85

FACT TABLES

70

CONFORMED DIMENSIONS

63m

DEFECTS INJECTED

1050

SA SITES

9

PROVINCES

Databricks · Delta Lake · Unity Catalog · dbt · Azure · MLflow · Power BI
2026-09-07

Disclaimer. Independent synthetic portfolio project. This document and the platform it describes contain no internal Vivo Energy, Engen, Shell or Vitol data. Every site, customer, employee, supplier, asset, coordinate, price, volume, margin and incident is randomly generated. Company and person names are invented; any resemblance to a real organisation or individual is coincidental and unintended. Site coordinates are town centroids plus random jitter and do not represent any real service station.

Contents

- | | |
|---|---------------------------------|
| 1. The brief | 7. Architecture |
| 2. What was built | 8. Decision support |
| 3. Data worth modelling on | 9. Machine learning |
| 4. Then it had to be broken | 10. Governance |
| 5. Cleansing that accounts for everything | 11. Running on Databricks |
| 6. Two defects it found in itself | 12. What I would do differently |

1. The brief

Build a data platform for a downstream energy business: a retail forecourt network, a commercial B2B book, primary distribution, LPG, lubricants, aviation, marine, loyalty and digital channels, EV and solar, asset maintenance, safety, and finance. Model it on South Africa. Make it real enough that the decisions taken on it would be defensible.

The additional constraint I set myself: **the data has to be dirty**. A platform demonstrated on clean data demonstrates almost nothing, because the difficult part of data engineering is not the join. It is deciding what to do with the row where `litres` reads `"1 234,5"`, the site code has a trailing space, and the same transaction arrived twice because the point-of-sale replayed its batch.

2. What was built

37.2m

FACT ROWS

85

FACT TABLES

70

CONFORMED DIMENSIONS

63m

DEFECTS INJECTED

1050

SA SITES

9

PROVINCES

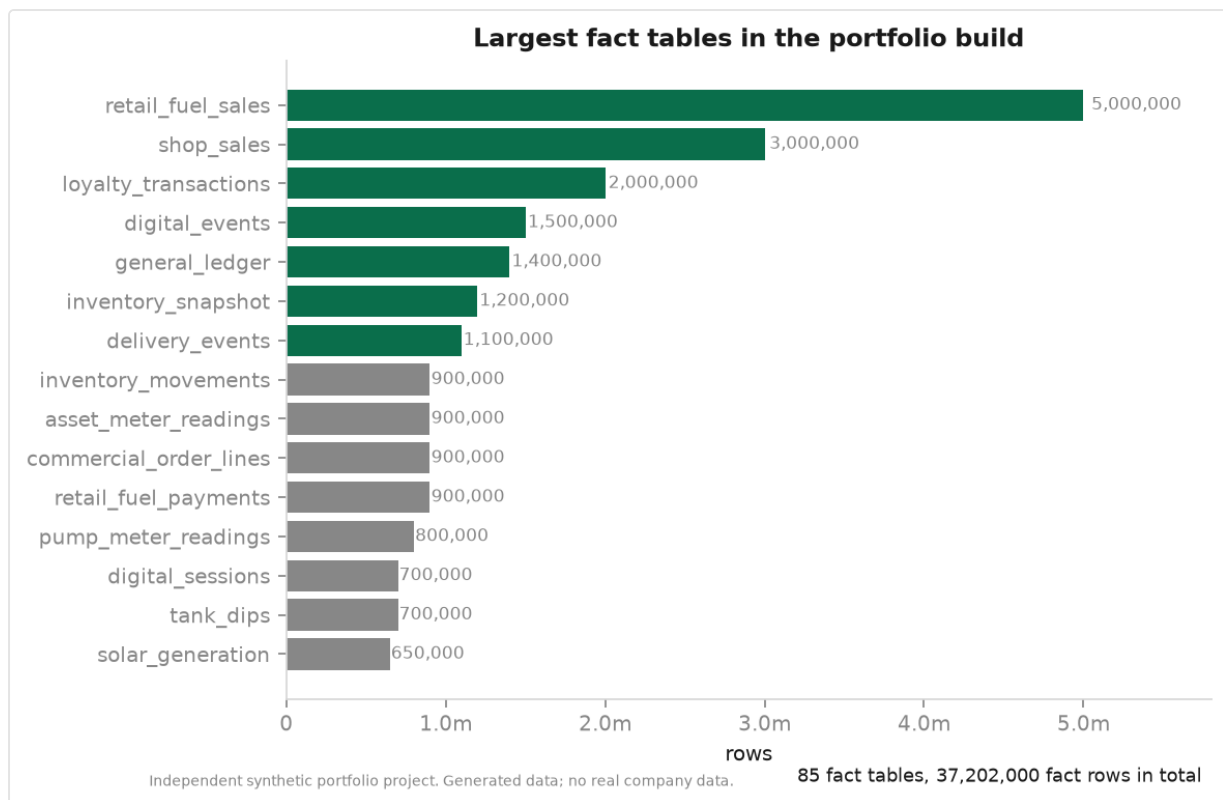


Figure 1. The largest fact tables produced by the portfolio build.

3. Data worth modelling on

Random numbers are not a dataset. If transaction timestamps are uniform, a demand forecast has nothing to learn and any accuracy figure it produces is meaningless. The generator therefore encodes the actual rhythms of the business: commute peaks, a Friday-heavy week, December holiday travel, LPG demand rising through winter, and the fact that the day *before* a public holiday is the strongest fuel day of the year.

South African pump prices are regulated and adjusted monthly, so the synthetic price is a per-month step function rather than per-transaction noise. Each site carries a `demand_index` combining urban class, national-route position and offer mix, so a Gauteng truck stop and a rural single-product site behave differently.

This took two attempts. The first version weighted which assets generated maintenance work orders but drew the work type independently, so the breakdown rate came out flat across asset age — 12%, 16%, 12%, 11% — and the predictive-maintenance model had nothing to find. After making breakdown probability depend on the asset's own condition, the rate rises 23% → 39% → 51% → 59% across age bands. Same row count; the difference is whether the dataset means anything.

4. Then it had to be broken

62,924,526 defects were injected across the 37,202,000-row build, modelled on the specific systems being simulated rather than sprinkled at random.

Defects injected into the landing zone.

DEFECT CLASS	COUNT
numeric delivered as text	56,600,000
locale decimal comma	1,697,958
whitespace padding	1,504,943
thousands separator	1,097,944
case inconsistency	738,361
sentinel string	474,245
numeric sentinel	207,191
null required field	187,192
duplicate rows	93,468
negative measure	86,613
truncated identifier	71,093
orphan foreign key	47,436
unflagged currency	41,590
epoch timestamp	31,612
future timestamp	23,742
derived value mismatch	12,256
unit of measure error	7,233
encoding mojibake	1,649

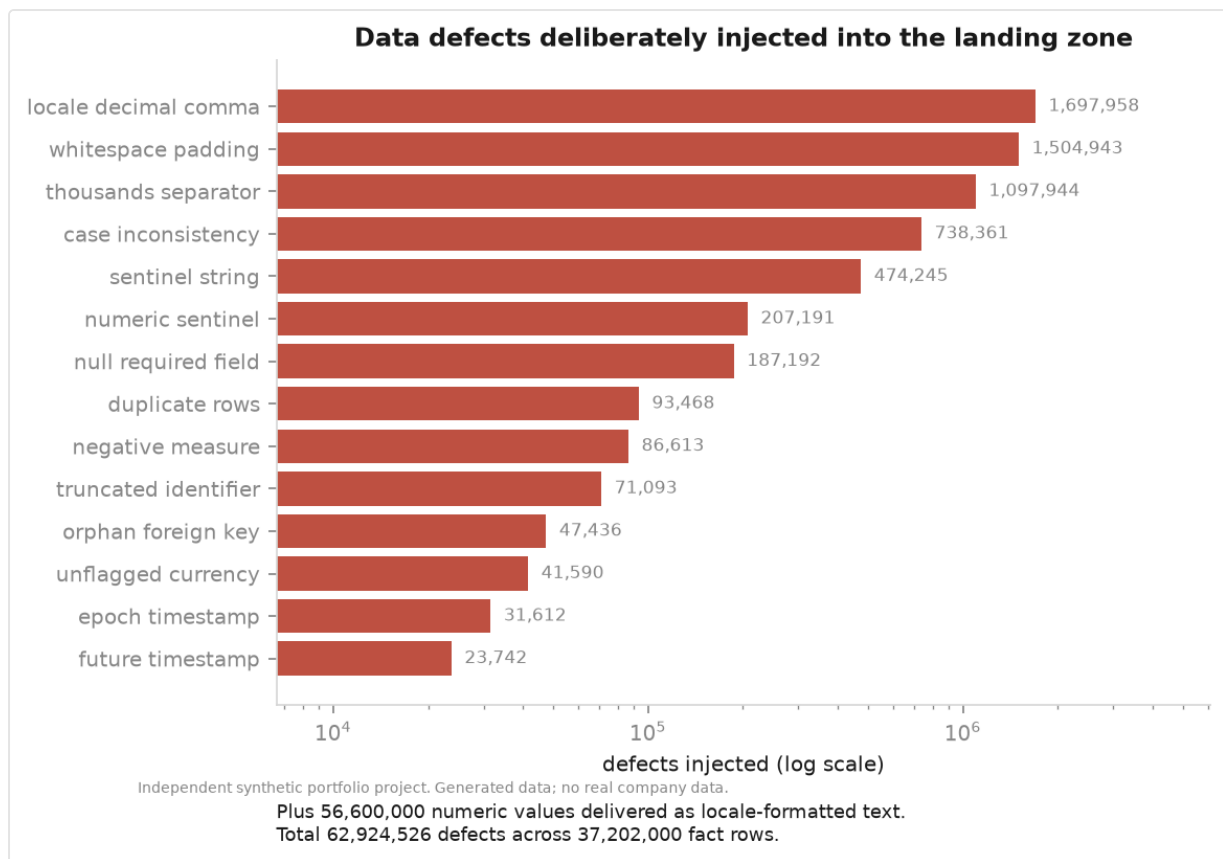


Figure 2. Defects deliberately injected into the landing zone, by class. Every one is counted at injection time, which is what makes the cleansing layer measurable rather than merely asserted.

Every defect is counted as it is injected and written to the build manifest. That is the design decision that matters: it converts cleansing from a claim into a measurement.

5. Cleansing that accounts for everything

167 generated staging models repair, validate and deduplicate. 122 quarantine models keep every rejected row alongside the rule that rejected it. The reconciliation that has to hold is simple:

```
raw rows = cleansed + quarantined + duplicates removed
```

A row that is neither cleaned nor rejected nor identified as a duplicate has been silently lost, and a silently lost row becomes an unexplainable variance three months later that nobody can trace.

The hardest single rule

Numeric parsing. A comma is ambiguous: in "1,234.56" it is a thousands separator, in "1234,56" it is the decimal mark. Getting this wrong does not throw an error — it produces a number wrong by three orders of magnitude that looks entirely plausible. The rule is positional, documented once, and applied identically in all 167 models, because the same value parsed two different ways in two tables is worse than not parsing it at all.

6. Two defects it found in itself

Both were caught by the platform's own controls, which is the entire reason for having them.

A macro that silently corrupted every ratio

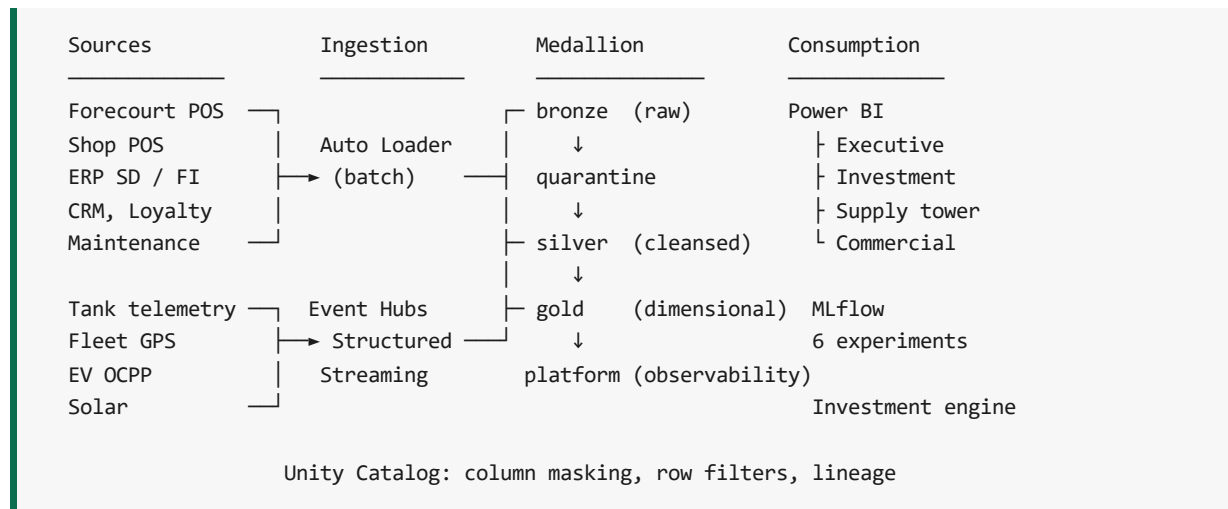
`safe_divide('a - b', 'c')` compiled to `a - (b / c)` because the macro did not parenthesise its arguments. Every margin percentage, utilisation rate and share-of-total in the project was wrong — not obviously wrong, just quietly incorrect. It was caught by a plausibility test asserting that product margin falls between -5 and 100 per cent. A `not_null` test would never have found it.

A general ledger that did not balance

Journal lines were posted independently, so total debits exceeded credits by 46%. No finance function would accept that. It was caught by the reconciliation controls, which evaluate double-entry as *data* rather than only as a test, and fixed by emitting journals as balanced pairs.

The lesson. `not_null` and `unique` catch structural faults. Business-plausibility and reconciliation controls catch the faults that reach a board pack.

7. Architecture



The medallion runs unchanged on DuckDB locally and on Databricks with Delta Lake. Cross-adapter differences are handled in a small set of dbt macros, so the models themselves have no environment-specific branching.

8. Decision support

`network_investment_scorecard` scores every site by combining trading performance, growth, non-fuel mix, reliability, downtime, safety and asset condition. Percentile ranks put raw figures and incident counts on a common scale, and the weights are an explicit management judgement stated in the SQL rather than a fitted parameter.

Every component is a visible column. A challenged recommendation can be answered with its inputs rather than with “the model said so”, which is the only version of this that survives an executive forum.

Ranking is not allocation

The investment engine turns the ranking into a capital plan: an exact bounded knapsack over site and intervention pairs under a fixed budget, with a minimum number of funded projects per province.

That provincial constraint exists because a pure return-maximising plan concentrates everything in Gauteng and the Western Cape. That is defensible financially and indefensible as a network strategy — divestment candidates that are the only site within 50 km score badly and should probably still be kept, because the volume does not transfer to a competitor's neighbour 200 km away.

9. Machine learning

Six MLflow experiments: demand forecast, predictive maintenance, customer churn, stock-out risk, late delivery, and unsupervised forecourt anomaly detection. Three rules applied to all of them.

Chronological splits, never random. A random split on time-series data leaks the future into training and produces an accuracy figure that will not survive contact with production.

No feature unknown at prediction time. The stock-out model deliberately excludes `days_of_cover`, even though the target is derived from it — the model would score near-perfectly by reproducing the definition and be worthless the moment the threshold changed.

A baseline that has to be beaten. The demand forecast reports its error against a seasonal-naive baseline — same weekday, last week — and prints a warning if it fails to beat it. A model that cannot beat “last Tuesday” should not be promoted, however good its R^2 looks.

The anomaly detection is unsupervised because there is no labelled fraud, and in a real downstream business there rarely is. The run is tagged explicitly: a high score means unlike this site's own history, not fraudulent. Most of what surfaces will be a miscalibrated meter or a badly closed shift, and that is still worth finding.

10. Governance

Unity Catalog column masks and row filters, mirrored by Power BI row-level security so a user sees the same rows whichever way they query.

The principle is that the default is restrictive: a new analyst sees masked customer names and no rows outside their region until someone deliberately grants more. Getting this the other way round is how platforms leak.

Two details worth calling out. Analysts get **no bronze access at all** — raw data has no conformed definitions, and any number taken from it will disagree with the dashboard. And the row-filter scope comes from a table rather than hard-coded group names, so adding a region to someone's remit is a data change, not a deployment.

11. Running on Databricks

The full medallion was deployed and executed on a Databricks workspace at portfolio scale. Bronze is generated in-platform rather than uploaded: pushing 37 million rows over a domestic connection takes hours, and Spark produces the same distributions on the cluster in minutes.

12. What I would do differently

The row-count plan drove the model

Starting from “we need 15 million rows” produced 85 fact tables, some of which exist mainly to carry volume. A real engagement would start from the decisions the business needs to make and let the model follow. The bus matrix would be shorter and better for it.

Uplift assumptions are asserted, not derived

“A convenience build lifts margin 22%” is a planning number I chose. In a real engagement it would come from the sites already converted — which the model can support, but which needs a before/after study rather than an assumption.

Anomaly detection has no feedback loop

The model produces a worklist; a production version would capture investigation outcomes and become supervised within a couple of quarters. Without that it degrades into an alert nobody opens.

Streaming is demonstrated rather than operated

The pipelines, watermarks and windowing are real, but I have not run them under sustained load or handled a poison message mid-stream. I would not claim production streaming experience on the strength of this.

Disclaimer. Independent synthetic portfolio project. This document and the platform it describes contain no internal Vivo Energy, Engen, Shell or Vitol data. Every site, customer, employee, supplier, asset, coordinate, price, volume, margin and incident is randomly generated. Company and person names are invented; any resemblance to a real organisation or individual is coincidental and unintended. Site coordinates are town centroids plus random jitter and do not represent any real service station.